

Lezione — CALC 02p: Fondamenta per il mapping (macchina, chip, numeri)

Corso: Calcolatori Elettronici T **Materiale:** dispensa 01 *Introduzione* (pp. 9, 30–36) · dispensa 02 *Mapping e decodifica* (pp. 2–13, 34–38) · dispensa 00 *Complementi ed esercizi di Reti Logiche*, esercizio 10 (pp. 68–74) **Prerequisiti:** nessuno. È la lezione da leggere **prima** di [[lezione_02_mapping_decodifica]] **Consegna servita:** nessuna direttamente; serve a leggere il testo della prova e la consegna **a)**

Perché esiste: la lezione 02 dava per scontati l'ambiente fisico, la notazione e il binario. Senza queste basi l'allineamento resta un'astrazione (sessione del 2026-10-08).

Si legge a **blocchi**: alla fine di ogni sezione numerata c'è una **Pausa**. Lì ti fermi e verifichi di saperlo ridire con parole tue, prima di passare alla sezione dopo.

Obiettivo

Davanti a una riga come «2 GB di EPROM mappata agli indirizzi bassi», sapere **quali oggetti fisici** ci sono sulla scheda, **quali fili** li collegano, e **quali numeri** (in binario e in esadecimale) delimitano la loro zona.

1. La macchina: una CPU, dei fili condivisi, tanti chip

Nel modello del corso, programma e dati stanno **in memoria**. La CPU non ha altro modo di leggerli o scriverli che **pilotare dei fili** secondo una sequenza temporale fissa, il **ciclo di bus** [fonte: 01 p. 30]. I fili sono di tre gruppi [fonte: 01 p. 31]:

- **indirizzi** BA[K..0]: la CPU vi scrive *chi* vuole raggiungere;
- **dati** BD[R..0]: vi viaggia il contenuto, in uscita (scrittura) o in entrata (lettura);
- **controllo**: READ/WRITE (nelle soluzioni MEMRD/MEMWR) dicono *cosa* vuole fare, più READY e INT che vedrai nei moduli 04–05.

Memorie e periferiche sono attaccate **tutte agli stessi bus** [fonte: 01 p. 32]. In una lettura la CPU mette l'indirizzo su BA e attiva MEMRD; il dispositivo giusto risponde mettendo il dato su BD. In una scrittura la CPU mette indirizzo e dato, e attiva MEMWR [fonte: 01 pp. 33–34].

Ogni indirizzo corrisponde a **un byte** [fonte: 02 p. 7]. Con 32 fili di indirizzo i numeri possibili sono 2^{32} , cioè 4 GB di byte raggiungibili: lo **spazio di indirizzamento** [fonte: 02 p. 2; 01 p. 35]. Il DLX ha 32 bit di indirizzo. Lo spazio *non* è la memoria installata: è il numero di caselle che la CPU sa nominare. Il progettista decide quali caselle corrispondono a un chip e quali restano vuote.

Il docente usa un'analogia: l'indirizzo serve a «distribuire merci», con WR = consegna e RD = preleva [fonte: 02 p. 3].

△ *Distinzione da tenere separata (errore trasversale 1):* **indirizzo** (su BA, dice *dove*) e **dato** (su BD, dice *cosa c'è lì*). 80000000h è un indirizzo, non il valore contenuto.

Pausa — Sai dire cosa passa su BA, cosa su BD e cosa su RD/WR durante una lettura?

2. Dentro un chip: K piedini di indirizzo e un decoder interno

Ogni dispositivo si presenta alla CPU con la stessa interfaccia: CS, A[K-1..0], RD, WR, D[...] [fonte: 02 p. 10]. Un esempio reale è l'EPROM **128K × 8** [fonte: 02 p. 11]: 128K celle da 8 bit. Ha i piedini d'indirizzo A0...A16, cioè 17, perché $128K = 2^7 \cdot 2^{10} = 2^{17}$. Ha i piedini dati D0...D7 e i comandi CE* (chip enable) e OE* (output enable). La RAM dello stesso taglio aggiunge WE* (write enable), perché si può anche scrivere [fonte: 02 p. 13].

La regola generale: un dispositivo con $n = 2^K$ byte ha **K** bit di indirizzo interni [fonte: 02 p. 35]. Al suo interno c'è un **decoder di II livello a K variabili**, che sceglie la cella [fonte: 02 p. 36]. Quel decoder l'ha già fatto il costruttore. Tu non lo progetti: colleghi ai piedini A[K-1..0] i K fili bassi del bus (BA[K-1..0], nel caso a 8 bit).

Nelle periferiche K è piccolo (es. 2); nelle memorie è grande (RAM da 128 KB → K = 17) [fonte: 02 p. 35].

EPROM e RAM: l'EPROM è non volatile e a sola lettura, e lì sta il codice all'avvio. La RAM è volatile, leggibile e scrivibile [fonte: 02 pp. 11, 13]. Per questo, nelle soluzioni, l'EPROM riceve solo RD e CS.

Pausa — Una RAM da 32 KB: quanti piedini A ha? (Risposta in fondo, *Prova tu* n. 4.)

3. Il decoder di I livello: il pezzo che progetti tu

Tutti i chip vedono lo stesso indirizzo, quindi serve qualcuno che dica a **uno solo** «tocca a te». Lo fa il **decoder di I livello**, una rete combinatoria che legge alcuni bit di BA e genera un segnale CS_X per ogni dispositivo [fonte: 02 p. 4].

L'esempio minimo del docente [fonte: 02 pp. 8-9] ha una CPU con 3 bit d'indirizzo (8 caselle, da 000 a 111) e due memorie A e B da 4 byte:

111	}	BA2 = 1 → A	CS_A = BA2
110			CS_B = BA2*
101			
100			
011	}	BA2 = 0 → B	BA1, BA0 → piedini A1, A0 di entrambe (II livello, dentro il chip)
010			
001			
000			

4 byte = 2^2 , quindi 2 bit interni (BA1, BA0). Resta 1 bit, BA2, per scegliere il chip. Questo è tutto l'esercizio d'esame, in miniatura: **i bit alti scelgono il chip, i bit bassi la cella**.

Notazione da leggere al volo [fonte: 00 pp. 69, 78]: - A13* = A13 **negato**, cioè vale 1 quando A13 = 0. Su un piedino, CE* indica un segnale attivo basso; - · = AND, + =

OR.

Così CS_RAM_1 = A15*·A13* si legge «A15 vale 0 e A13 vale 0» [fonte: 00 p. 71].

Pausa — Nell'esempio sopra, con indirizzo 101, quale CS è attivo, e quale cella del chip si legge?

4. Perché il CS dev'essere uno solo alla volta: il 3-state

Le uscite dati di tutti i chip sono sullo **stesso** filo di BD. Questo funziona grazie al **driver 3-state** [fonte: 02 p. 5]. Con OE = 1 l'uscita copia l'ingresso (0 o 1). Con OE = 0 l'uscita va in **Z**, alta impedenza: il chip si scollega elettricamente dal filo.

La dispensa chiede: due driver sullo stesso filo, cosa bisogna garantire [fonte: 02 p. 6]? **Che non siano mai abilitati insieme.** Se uno spinge 1 e l'altro 0, U non ha un valore logico significativo. Da qui la regola d'esame: i CS devono essere **mutuamente esclusivi**. È il punto (f) della procedura nel prontuario.

Pausa — Perché basta che un solo dispositivo abbia OE attivo, e gli altri possono restare collegati?

5. I numeri: binario, potenze di due, esadecimale

Questa sezione è uno strumento di calcolo: la dispensa lo usa senza spiegarlo. Gli esempi però sono quelli delle sue tabelle.

Binario. Ogni posizione vale il doppio di quella alla sua destra. Su 4 bit i pesi sono 8 4 2 1, e il valore è la somma dei pesi dove c'è un 1: 1100 = 8 + 4 = 12; 0110 = 4 + 2 = 6. **Verifica sempre** rifacendo la somma: un bit letto male sposta un dispositivo di gigabyte (errore trasversale 2: far quadrare i numeri).

Potenze di due e unità. 1 K = 2^{10} = 1024; 1 M = 2^{20} ; 1 G = 2^{30} . Quindi 20 bit fanno 1 MB e 32 bit fanno 4 GB [fonte: 02 p. 2]. Per trovare i bit interni K di una taglia, scrivila come potenza di due:

Taglia	Potenza	K (bit interni)
2 KB	$2^1 \cdot 2^{10}$	11
8 KB	$2^3 \cdot 2^{10}$	13
128 KB	$2^7 \cdot 2^{10}$	17
64 MB	$2^6 \cdot 2^{20}$	26
512 MB	$2^9 \cdot 2^{20}$	29
1 GB	2^{30}	30
2 GB	2^{31}	31

Esadecimale. Una cifra hex = **4 bit esatti**, quindi si converte un gruppo alla volta (il suffisso h indica il formato):

0 0000	4 0100	8 1000	C 1100
1 0001	5 0101	9 1001	D 1101
2 0010	6 0110	A 1010	E 1110

3 0011 7 0111 B 1011 F 1111

Il docente scrive le tabelle proprio così, a gruppi di 4 [fonte: 00 p. 69]: 0010 0111 1111 1111 (27FFh). Un indirizzo DLX ha 32 bit, quindi 8 cifre hex: 80000000h = 1000 0000 ... 0000, cioè solo BA31 = 1.

Due regole che userai sempre [fonte: 02 p. 38]: - **multiplo di $2^K \Leftrightarrow$ ultimi K bit a zero**. Pari = ultimo bit 0; multiplo di 8 = ultimi tre a 0; multiplo di 16 = ultima cifra hex 0; multiplo di 64K = ultime 4 cifre hex a 0; - **fine di un blocco = ultimi K bit a 1**: 1FFFh = 0001 1111 1111 1111 chiude un blocco da 8K che parte da 0000h.

Pausa — Converti 1FFFh e 2000h in binario e guarda cosa cambia passando dall'uno all'altro.

6. Un esempio svolto dal docente, letto con queste basi

Esercizio 10 [fonte: 00 pp. 68-69]: bus indirizzi a 16 bit (64K caselle, da 0000h a FFFFh) e bus dati a 8 bit. 12K di RAM in basso, 16K di EPROM in alto.

- 12K non è una potenza di due e va spezzato in chip che lo sono. Il testo non impone taglie: la scomposizione minima sarebbe **8K + 4K** (12 = 1100 in binario), ma il docente sceglie **8K + 2K + 2K**, probabilmente per mostrare più bit di decodifica. Regola: se il testo dà i chip disponibili si usano quelli, altrimenti le potenze di due di N in binario. I chip si dispongono a partire da 0: RAM_1 0000h–1FFFh (8K), RAM_2 2000h–27FFh (2K), RAM_3 2800h–2FFFh (2K). Ognuno parte da un multiplo della propria taglia: è l'**allineamento** della lezione 02.
- 16K di EPROM «in alto» finiscono a FFFFh e partono 16K prima: C000h–FFFFh.
- Segnali di decodifica: CS_RAM_1 = $A_{15} \cdot A_{13}$, CS_RAM_2 = $A_{15} \cdot A_{13} \cdot A_{11}$, CS_RAM_3 = $A_{15} \cdot A_{13} \cdot A_{11}$, CS_EEPROM = A_{15} .

Leggi CS_RAM_1 con la sezione 3. A_{15} dice «metà bassa», quindi non EPROM. A_{13} dice «non le RAM da 2K», che stanno tutte a 2xxxh, dove $A_{13} = 1$. Il docente non guarda A_{14} , e questo ha una conseguenza: RAM_1 risponde anche a 4000h–5FFFh [fonte: 00 p. 71]. È la **decodifica semplificata** con le sue **repliche** [fonte: 00 p. 74], cioè la Decisione 3 della lezione 02.

Pausa — Ripercorri da solo perché CS_RAM_2 ha bisogno di A_{11} e CS_RAM_1 no.

7. Leggere un testo d'esame

Testo d'esempio dalla dispensa [fonte: 01 p. 9, prova del 05/09/2024]: «processore DLX dotato di 2 GB di EPROM mappata agli indirizzi bassi e 1 GB di RAM mappata a partire dall'indirizzo 80000000h», più quattro porte di input da 8 bit.

Tradotto con le basi di questa lezione (ragionamento mio: la soluzione di questa prova non è nell'archivio):

- **DLX** → BA a 32 bit, spazio 00000000h–FFFFFFFFh (4 GB);
- **2 GB di EPROM agli indirizzi bassi** → 2 GB = 2^{31} byte, da 0: 00000000h–7FFFFFFFFh, cioè tutta la zona con BA31 = 0;

- **1 GB di RAM da 80000000h** → $1 \text{ GB} = 2^{30} = 400000000 \text{ h}$ byte: 80000000h–BFFFFFFFh, cioè BA31 = 1, BA30 = 0. È allineata: 80000000h ha gli ultimi 30 bit a zero;
- **porte di input** → dispositivi piccoli, da mappare nello spazio libero (C0000000h in su). Handshake e interrupt sono materia dei moduli 04–05.

Quello che manca per scrivere la soluzione vera è il **bus dati a 32 bit** del DLX, con i banchi e i segnali BE: è la Decisione 2 della lezione 02. Qui, come nella dispensa fino a p. 34, abbiamo supposto un bus dati a 8 bit.

Prova tu

1. 1010, 0100, 1110 in decimale: quali sono multipli di 4?
2. 2800h in binario.
3. Un chip da 2K può partire da 2800h?
4. Quanti piedini A ha una RAM da 32 KB? E una da 512 KB?
5. Ultimo indirizzo di un blocco da 1 GB che parte da 00000000h.
6. Nell'esercizio 10, con indirizzo 2400h, quale CS si attiva?

Soluzioni

1. 10, 4, 14. Solo 0100 (= 4) finisce con 00.
2. 0010 1000 0000 0000.
3. Sì. $2K = 2^{11}$, e gli 11 bit bassi di 2800h (da A10 a A0) sono tutti 0; il bit A11 = 1 sta sopra.
4. $32K = 2^{15} \rightarrow 15$ piedini (A14..A0); $512K = 2^{19} \rightarrow 19$.
5. 3FFFFFFFh: 30 bit bassi a 1.
6. 2400h = 0010 0100 0000 0000: A15 = 0, A13 = 1, A11 = 0 → CS_RAM_2.

Riepilogo

Cosa succede sui fili quando la CPU legge il byte all'indirizzo X? La CPU mette X su BA e attiva MEMRD. Il decoder di I livello guarda i bit alti di X e attiva un solo CS. Quel chip usa i bit bassi (decoder di II livello, interno) per scegliere la cella e mette il contenuto su BD. Tutti gli altri chip restano in Z.

Come si passa da «un chip da N byte» ai bit da decodificare? $N = 2^K$: i K bit bassi vanno al chip (piedini A[K-1..0]), quelli sopra (α) sono affare tuo. In più, un chip allineato ha α fisso su tutta la sua zona.

Perché le soluzioni scrivono gli indirizzi in esadecimale? Perché ogni cifra sono 4 bit esatti. Da C000h leggi subito 1100 0000 0000 0000, cioè quali bit di BA sono fissi e quali liberi: è la stessa informazione del binario, in un quarto dello spazio.

Prossimo passo: riprendi [[lezione_02_mapping_decodifica]] dalla Decisione 1.